Chapter 9
Scheduling: Proportional Share

Dong Dai
CIS/UD
dai@udel.edu

Proportional Share

- Fair-share scheduler
  - guarantee that each job obtain a certain percentage of CPU time.
  - ignore turnaround time or response time
- Fairness Metrics
  - the time the first job completes divided by the time that the last job completes
- Three ideas
  - Lottery scheduling
  - Stride scheduling
  - Linux CFS

1. Lottery Scheduling

- Tickets
  - The share of a resource that a process should receive
  - A, B
    - A has 75 tickets -> 75% CPU
    - B has 25 tickets -> 25% CPU
  - holding a lottery every time slice
    - Know total number of tickets (100)
    - Pick a winning ticket (random 0-99)
    - 0-74 runs A
    - 75-99 runs B

Example

- Winning tickets

```
63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43  0 49 12
```

- Resulting schedule

```
A     A  A     A  A  A  A  A  A     A     A  A  A  A  A  A
   B        B                    B     B
```

- Results
  - B runs 4 times out of 20 time slices (20%)
  - Close to 25% but not perfect.

Tickets

- Ticket currency
  - Currency allows a user with a set of tickets to allocate tickets among their own jobs in whatever currency they would like; the system then automatically converts said currency into the correct global value.
- Ticket transfer
  - a process can temporarily hand off its tickets to another process.
- Ticket inflation
  - a process can temporarily raise or lower the number of tickets it owns.
  - Only make sense in a cooperative environment

Implementation

- Total number of tickets for getting winning ticket
- Iterate list and add each ticket to counter
- Compare counter and winning ticket

```
// counter: used to track if we've found the winner yet
int counter = 0;

// winner: use some call to a random number generator to
//          get a value >= 0 and <= (totaltickets - 1)
int winner = getrandom(0, totaltickets);

// current: use this to walk through the list of jobs
node_t *current = head;
while (current) {
    counter = counter + current->tickets;
    if (counter > winner)
        break; // found the winner
    current = current->next;
}
// 'current' is the winner: schedule it...
```

Stride Scheduling

- Lottery scheduling statistically guarantees fairness.
  - it occasionally will not deliver the exact right proportions, especially over short time scales.
- a deterministic fair-share scheduler?
  - Stride Scheduling
- Idea is simple
  - Each job has a stride, inverse in proportion to the number of tickets it has.
  - every time a process runs, we will increment a counter for it (called pass value) by its stride to track its global progress.
  - at any given time, pick the process to run that has the lowest pass value so far

```
curr = remove_min(queue);   // pick client with min pass
schedule(curr);             // run for quantum
curr->pass += curr->stride; // update pass using stride
insert(queue, curr);        // return curr to queue
```

Example

- Stride values of A, B, C are 100, 200, 40
- Init pass values for all are 0

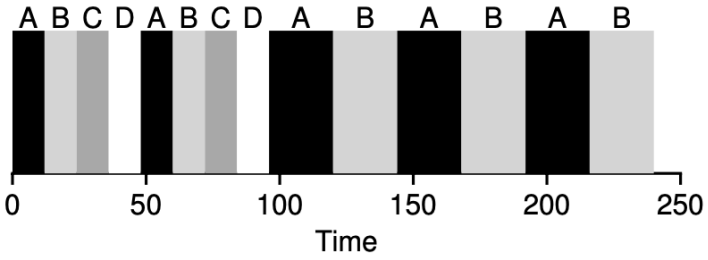
Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

- Issues?
  - Can not handle new jobs very well.

Linux CFS

- Completely Fair Scheduler
  - introduced in Linux kernel version 2.6.23
  - CFS is still the default process scheduler in modern Linux kernels.
- virtual runtime (vruntime)
  - As each process runs, it accumulates vruntime
  - each process's vruntime increases in proportion with physical (real) time
  - When a scheduling decision occurs, CFS will pick the process with the lowest vruntime to run next.
- ? When to stop, when to schedule?

Control Parameters



- sched_latency
  - how long one process should run before considering a switch
  - 48 ms, shared by all active processes. 4 processes mean each process runs 12 ms
  - Too many processes? -> too small time slice -> too many context switch…
- min_granularity
  - 6 ms, never set the time slice of a process to less than this value
- Leverage periodic timer interrupt

Niceness

- Introduce priority into CFS
  - give some processes a higher share of the CPU
  - Not via ticket, but a classic UNIX mechanism known as the nice level of a process.
- Nice (-20 – +19), default 0
  - Positive values imply lower priority
  - Negative values imply higher priority
  - Too nice means less priority?...

```
static const int prio_to_weight[40] = {
    /* -20 */ 88761, 71755, 56483, 46273, 36291,
    /* -15 */ 29154, 23254, 18705, 14949, 11916,
    /* -10 */  9548,  7620,  6100,  4904,  3906,
    /*  -5 */  3121,  2501,  1991,  1586,  1277,
    /*   0 */  1024,   820,   655,   526,   423,
    /*   5 */   335,   272,   215,   172,   137,
    /*  10 */   110,    87,    70,    56,    45,
    /*  15 */    36,    29,    23,    18,    15,
};
```

Niceness

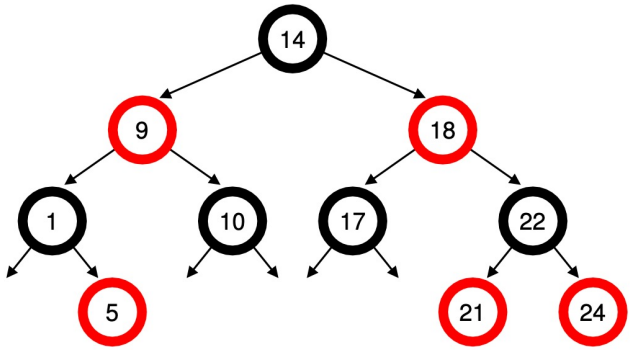
- Niceness -> Weight
  - weights allow us to compute the effective time slice of each process

$$\texttt{time_slice}_k = \frac{\texttt{weight}_k}{\sum_{i=0}^{n-1} \texttt{weight}_i} \cdot \texttt{sched_latency}$$

  - A (-5), B(0)
  - Weight(A) = 3121, Weight(B) = 1024
  - Time-slice(A) = $\frac{3}{4}$; Time-slice(B) = $\frac{1}{4}$
- Vruntime updates
  - scales it inversely by the weight of the process
  - Weight_0 = 1024

$$\texttt{vruntime}_i = \texttt{vruntime}_i + \frac{\texttt{weight}_0}{\texttt{weight}_i} \cdot \texttt{runtime}_i$$

  - A's vruntime will accumulate at one-third the rate of B's

O(LogN)

- Recent studies have shown that scheduler efficiency is surprisingly important; specifically, in a study of Google datacenters, Kanev et al. show that even after aggressive optimization, scheduling uses about 5% of overall datacenter CPU time
- CFS has O(LogN) complexity
  - by keeping processes in a red-black tree
  - only running (or runnable) processes
  - vruntime: 1, 5, 9, 10, 14, 18, 17, 21, 22, and 24
  - Processes are ordered in the tree by vruntime
  - Get_minimal -> O(logn)
  - Insert/Delete -> O(logn)